Natural Language Processing With Deep Learning

Course Project Review

SLM

Aarya Upadhya (PES1UG23AM006)
Anshull M Udyavar (PES1UG23AM057)

Heterogeneous Memory Augmented Reasoning for KG completion

- Knowledge graph Embeddings fail silently on Hard triples, they rank wrong and move in with no mechanism to learn from the failure. We have built a memory augmented multi agent framework that diagnoses the KGE failures, accumulates the validated reasoning experience through episodic memory writeback, and recover 98.5% of hard triples where RotatE achieves 0.0 Hits@1 without an retraining, without API dependence and without Modifying mode weights during inference runs
- Hypothesis: ***"Can a richer schema from Teacher-Student based distillation reasoning on top of KGE prior solve Hard Triples ?"***

What is unique about this project ?

- **KGE failure as a reasoning signal, not a training signal.** Every prior approach to improving knowledge graph embeddings retrains or augments the embedding model itself. This work treats ComplEx as a fixed diagnostic oracle. When it fails, an agentic layer reasons about *why* the geometry failed for that specific query — what structural property of the graph the embedding space could not capture — rather than simply predicting a better answer. The failure mode becomes the input, not the loss.
- **Quality-gated writeback, not static retrieval.** Standard RAG systems retrieve from a fixed document store. This system writes back to episodic memory only when reasoning is validated. A prediction that is correct but ungrounded — high confidence, low reasoning quality, what the lucky rate diagnostic identifies — is explicitly penalized and not stored. Memory grows only from wins the system genuinely understood.
- **Grounded scoring as a process reward model.** During training, a grounded scorer uses the known correct answer to identify which relations in the graph actually separate the true tail from the predicted one. This is structurally identical to Process Reward Models in RLHF, where each reasoning step is evaluated with access to the ground truth rather than only the final outcome. The novelty is applying this step-level grounding to relational path discrimination in a knowledge graph.
- **Episodic memory satisfying DeChant's four safety principles by construction, plus a fifth.** DeChant (2025) independently identified four necessary properties for safe episodic memory in AI agents: interpretability, user controllability without retraining, detachability from model weights, and prevention of agent self-editing of past records. This system satisfies all four as emergent architectural properties, not post-hoc additions — the memory is structured JSON, fully inspectable, detached from KGE weights, and write-protected from the agents themselves. Beyond DeChant, this work identifies a fifth risk specific to continual learning: **Memory Ossification**, where early low-quality episodes anchor and distort all future reasoning. This is addressed through quality-gated writeback and plateau-triggered memory auditing, neither of which appears in prior episodic memory literature

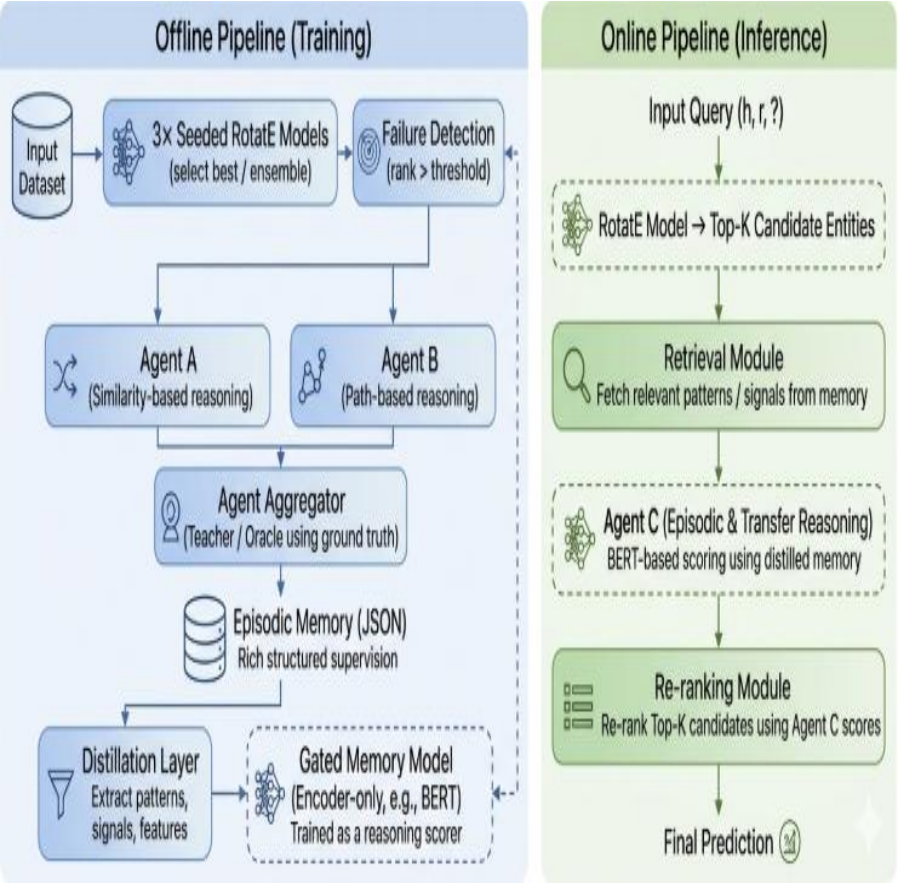
Dataset

1. ***UMLS***
   - Closed world dataset, Dataset developed/used by **University of Missouri–St. Louis**
   - Small-scale model validation
   - Baselien comparison.
2. ***CODEX-S***
   - Modern Benchmark for KGC Completion (Extracted from Wikipedia + Wikidata)
   - Dataset Statistics:
     - 2304 triples
     - 42 different relation's
     - 36k Triples
     - Supports both Link prediction and Triple classification

Natural Language Processing with Deep Learning

Architecture (Training)

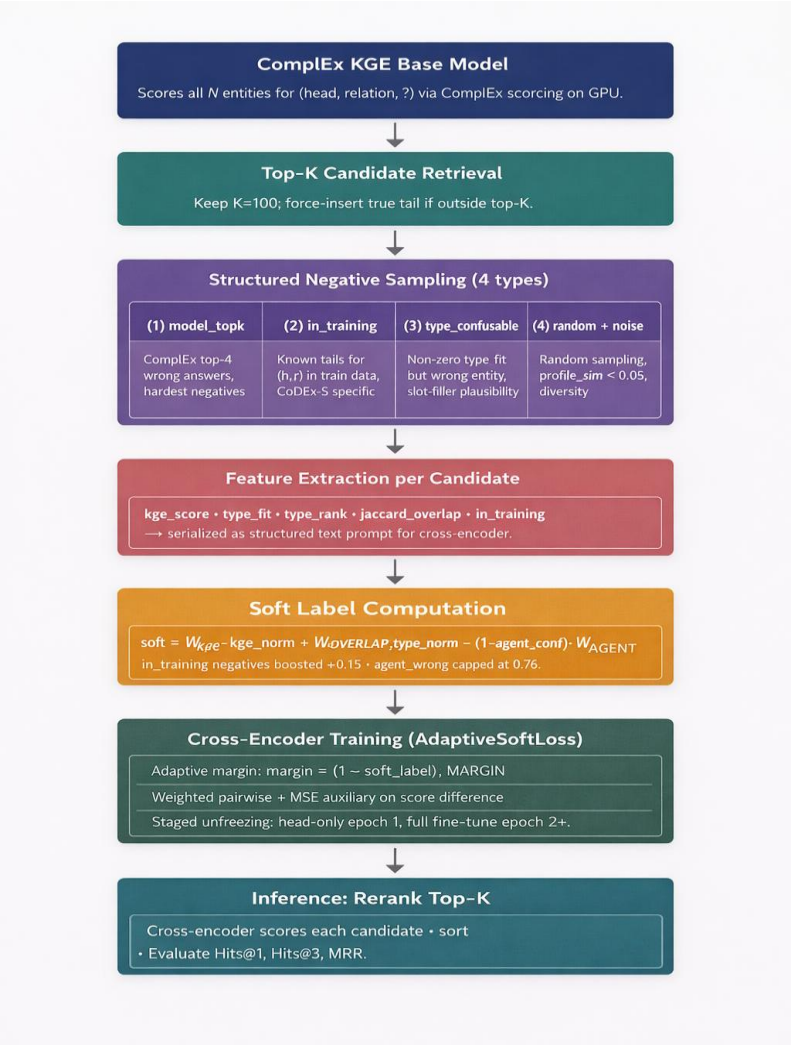
1. ***KGE Failure Detector***: ComplEX and RotatE on all candidates and then find the hard failures that become the input to the agents, 3 seeded runs for each has been done
2. ***Structured Failure Context Builder***: For each hard failure we create a summary via local subgraph and then separating signal and separating Profile and hop type . The context is injected as prompt as natural language we follow a REACT based prompting network with context engineering to enrichen the data with episodic writeback in later training , the Framework contains Agent A , B and a Aggregator.
3. ***Grounded scorer and Aggregator***: helps getting a post hoc analysis on why agents either went wrong or got the correct answer helps us distil correct information and understand how the Graph exists in Leyman terms with no Geometry, the ground scores is used to make a decision on which agent makes sense
4. The models Output is mainly used for 2 main reasoning for richer reasoning on each hard triple and Understand why the model choose that Agent to solve the Hard problem
5. **LLM : (Agent A + B + Aggregator uses : Qwen 7B Instruct**



Natural Language Processing with Deep Learning

Architecture (Inference)

1. ***Stage 1***: KGE (Complex / RotatE) this acts as a prior to help us get the top k candidates
2. ***Stage 2***: We construct Hard negatives, by inserting the models reasoning failures and KGE ambiguity to make sure the re-ranking model does not memorize
3. ***Stage 3***: Structured Feature Encoding: We make sure to calculate the Jaccard Overlap and other features that would help in re-ranking sourced from the Training Distillation process .
4. ***Stage 4***: Soft Supervision (make sure they are not binary labels)
5. ***Stage 5***: Cross Encoder training
   - Adaptive margin
   - Weighted pairwise
   - Staged Unfreezing
   - **(Re-ranking Model : cross-encoder/ms-marco-MiniLM-L-6-v2)**
6. ***Stage 6*** : Reranking



Evaluation Metrics

1. Hits@k (Ranking Accuracy)
   - Checks if the correct answer is in the top K predictions

$$\text{Hits@K} = \frac{1}{|Q|} \sum \mathbf{1}[\text{rank}(t^*) \leq K]$$

2. Mean Reciprocal Rank
   - Rewards earlier ranks more strongly

$$\text{MRR} = \frac{1}{|Q|} \sum \frac{1}{\text{rank}(t^*)}$$

3. Our Diagnostic Metric (Lucky rate vs Grounding Rate)
   - We analyse why a prediction is correct, not only why its correct
     - Grounded -> BERT already ranked correct answer in Top T
     - Lucky -> BERT ranked it low , either Parametric memory (In LLM's) or KGE pushed it Higher (in Inference)
     - Let S = queries where combined model predicts rank = 1

$$\text{LuckyRate} = \frac{|S_{\text{lucky}}|}{|S|}, \quad \text{GroundRate} = \frac{|S_{\text{grounded}}|}{|S|}$$

$$\text{LuckyRate} + \text{GroundRate} = 1$$

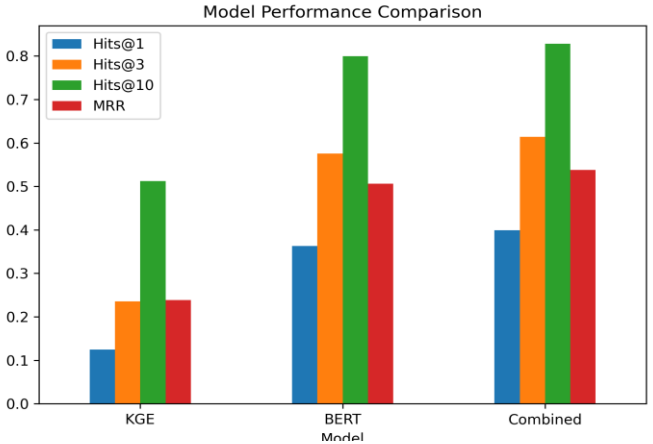
Natural Language Processing with Deep Learning

Evaluation Results (CODEX-S)

CODEX_S Results (Teacher Distillation Phase)

Main Performance table

Model	Hits@1	Hits@3	Hits@10	MRR
KGE (RotatE)	0.1247	0.2352	0.5120	0.2382
BERT	0.3632	0.5755	0.7998	0.5062
Combined	0.3993	0.6138	0.8282	0.5381

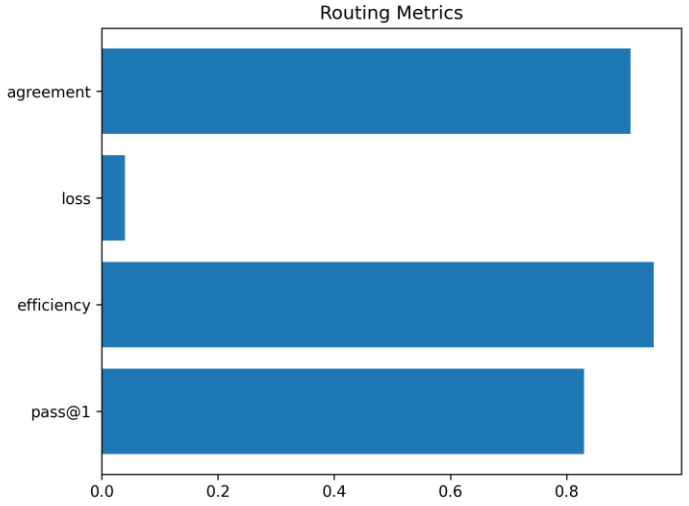


Diagnostic Table

Metric	Value
Ground Rate	0.934
Lucky Rate	0.066
Improved Queries	685
Unchanged	130
Worsened	99

Routing Metrics

Metric	Value
pass@1_routed	0.8317
pass@2_total	0.8713
pass@2_grounded	0.8713
pass@2_lucky	0.0000
routing_efficiency	0.9545
routing_loss	0.0396
agreement_rate	0.9109

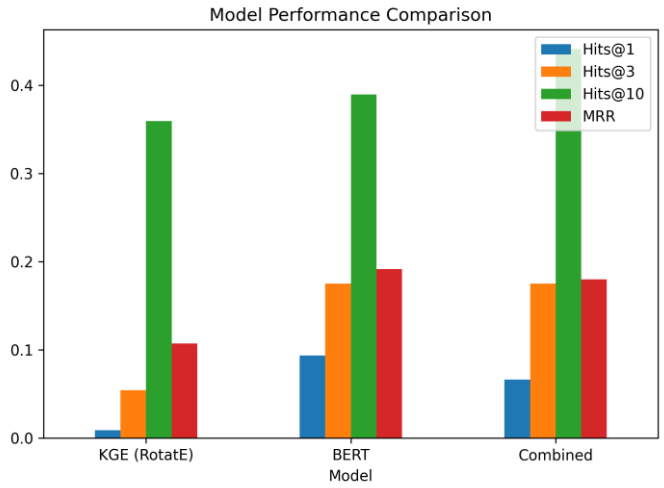


Natural Language Processing with Deep Learning

Evaluation Results (UMLS)

Main Performance table

Model	Hits@1	Hits@3	Hits@10	MRR
KGE (RotatE)	0.0091	0.0544	0.3595	0.1074
BERT	0.0937	0.1752	0.3897	0.1917
Combined	0.0665	0.1752	0.4411	0.1798

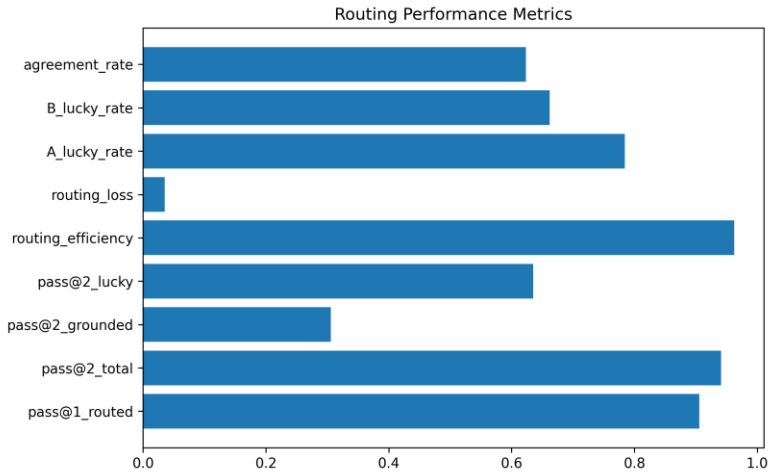


Diagnostic Table

Metric	Value
Improved	169
Unchanged	8
Worsened	154
Ground Rate	0.909
Lucky Rate	0.091

Routing Metrics

Metric	Value
pass@1_routed	0.9059
pass@2_total	0.9412
pass@2_grounded	0.3059
pass@2_lucky	0.6353
routing_efficiency	0.9625
routing_loss	0.0353
A_lucky_rate	0.7843
B_lucky_rate	0.6623
agreement_rate	0.6235



THANK YOU

PES UNIVERSITY

CELEBRATING 50 YEARS